

Express를 활용한 서버 구조 개선

□ Express란 무엇인가?

○ 개념 정의

- Express는 Node.js를 위한 가장 널리 사용되는 웹 프레임워크
- http 모듈보다 더 쉽고 간결하며 빠르게 웹 서버를 구성할 수 있음
- REST API, 미들웨어, 라우팅 등 구조화된 웹 개발에 적합

○ Express가 필요한 이유

- 라우팅, 에러 처리, 요청 본문 처리 등이 매우 쉬움
- 프로젝트가 커져도 모듈화로 구조 관리가 수월
- 다양한 미들웨어와의 조합으로 확장성이 뛰어남

○ 주요 특징

- 미들웨어 기반 구조
- 빠르고 간결한 라우팅 처리
- 다양한 외부 라이브러리와 연동 용이 (ejs, cors 등)

□ Express 기본 사용법

○ 설치하기

- 명령어: `npm install express`

```
● (base) ggoreb@GGoRebui-MacBookAir ex % npm install express
added 66 packages, and audited 68 packages in 1s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ (base) ggoreb@GGoRebui-MacBookAir ex %
```

○ 기본 서버 만들기

- app.js

```
import express from 'express';

const app = express();

app.get('/', (req, res) => {
  res.status(200).type('html').send('<h1>Hello, Express!</h1>');
});

app.listen(3000, () => {
  console.log('http://localhost:3000 실행 중');
});
```



코드 줄	설명
<code>import express from 'express';</code>	express 모듈을 불러옴 (ES 모듈 방식)
<code>const app = express();</code>	Express 앱 인스턴스를 생성 (이게 서버 역할)
<code>app.get('/', ...)</code>	/ 경로에 GET 요청이 왔을 때 실행할 콜백 함수 등록
<code>res.status(200) .type('html').send(...)</code>	상태 코드 200, MIME 타입 text/html로 응답을 보냄
<code>app.listen(3000, ...)</code>	3000번 포트에서 서버를 실행하며 실행 완료 시 콘솔 메시지 출력

□ 미들웨어 이해하기

○ 개념 정의

- 요청과 응답 사이에서 동작하는 함수
- 요청을 가로채서 공통 로직 수행 가능 (예: 로깅, 인증)

○ 미들웨어 작성 예시

- middleware/logger.js

```
export function logger(req, res, next) {  
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);  
  next(); // 다음 미들웨어로 이동  
}
```

- app.js

```
import express from 'express';  
import { logger } from './middleware/logger.js';  
  
const app = express();  
  
app.use(logger);  
  
app.get('/', (req, res) => {  
  res.status(200).type('html').send('<h1>Hello, Express!</h1>');  
});  
  
app.listen(3000, () => {  
  console.log('http://localhost:3000 실행 중');  
});
```



```
○ (base) ggoreb@GGoRebui-MacBookAir 03 % node app.js  
http://localhost:3000 실행 중  
[2025-05-16T07:21:31.478Z] GET /user/nodejs
```

□ 라우팅(Routing)이란?

○ 정의

- URL 경로에 따라 다른 응답으로 처리

ex) `/user` 요청과 `/product` 요청에 대해 각각 다른 화면이나 데이터를 응답

○ 기본 구조 예시

```
app.get('/user', (req, res) => {  
  res.send('사용자 페이지');  
});
```

```
app.get('/product', (req, res) => {  
  res.send('상품 페이지');  
});
```

○ 기본 http 모듈과 Express 비교

항목	기본 http 모듈	Express
라우팅 처리	if/else로 직접 분기	<code>app.get()</code> , <code>app.post()</code> 등으로 간결
응답	<code>writeHead</code> + <code>end</code> 필요	<code>res.send()</code> , <code>res.json()</code> 등 자동 처리
파일 분리 및 모듈화	직접 구현해야 함	<code>Router</code> 객체로 역할 분리 가능
미들웨어	직접 구현	<code>app.use()</code> 로 손쉽게 등록

라우팅 모듈화와 관리

□ 라우팅을 분리해야 하는 이유

- 라우트 수가 많아지면 한 파일로 관리하는 어려움
- 역할에 따라 파일을 나누고 기능별 모듈화가 필요

□ Express Router 객체

- Express에서 라우팅 기능을 독립된 모듈로 제공하는 객체
- `express.Router()`로 생성
- 미들웨어, 경로, 하위 라우트 등을 모듈처럼 관리 가능

[실습]

1. 파일 구성

```
project/  
├─ routes/  
│   └─ user.js  
└─ app.js
```

2. 라우터 모듈 - routes/user.js

```
import express from 'express';  
const router = express.Router();
```

```
// GET /user/
```

```
router.get('/', (req, res) => {  
  res.send('사용자 목록을 보여줍니다.');
```

```
// GET /user/profile
```

```
router.get('/profile', (req, res) => {  
  res.send('사용자 프로필 페이지입니다.');
```

```
// POST /user/register
```

```
router.post('/register', (req, res) => {  
  res.send('사용자 등록이 완료되었습니다.');
```

```
export default router;
```

3. 라우터 등록 및 실행 - app-route.js

```
import express from 'express';
import userRouter from './routes/user.js';

const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// /user로 시작하는 요청은 userRouter에게 위임
app.use('/user', userRouter);

app.get('/', (req, res) => {
  res.send('홈 페이지입니다.');
```

```
});

app.listen(3000, () => {
  console.log('http://localhost:3000 서버 실행 중');
```

```
});
```

4. 동작 확인

요청	설명	테스트 주소
GET /user	사용자 목록 확인	http://localhost:3000/user
GET /user/profile	사용자 프로필	http://localhost:3000/user/profile
POST /user/register	사용자 등록	API Tester로 확인

※ 경로 파라미터

- `/:name`처럼 경로 중에 `:` 기호가 붙은 부분은 동적으로 변하는 값
 - `/user/nodejs`, `/user/park`처럼 사용자 이름을 URL에 포함시켜 서버에서 해당 값을 추출
 - Express는 `req.params.키이름` 형태로 해당 값을 자동으로 추출

- 경로 파라미터 (`/user/:name`)
 - 리소스를 식별하는 데 사용
 - URL 구조에 포함됨 (정적인 느낌)
 - 예: `/user/nodejs`

- 쿼리스트링 (`/user?name=nodejs`)
 - 필터링, 옵션 설정에 사용
 - `req.query.키이름`으로 접근
 - 예: `/user?name=nodejs`

□ 경로 파라미터 여러 개 사용하기

- 예: `/user/:team/:name`
 - `/user/dev/nodejs` → 팀 이름과 사용자 이름 모두 추출 가능
 - `req.params.team`, `req.params.name` 사용

```
router.get('/:team/:name', (req, res) => {  
  const { team, name } = req.params;  
  res.send(`${team}팀의 ${name}님 환영합니다.`);  
});
```

□ 경로 파라미터의 위치와 충돌 주의

- 아래처럼 작성하면 `/user/profile` 보다 `/:name` 이 먼저 실행됨

```
// 잘못된 순서  
router.get('/:name', ...); // 먼저 정의됨  
router.get('/profile', ...); // 절대 도달되지 않음
```

- 해결: 고정 경로 먼저, 동적 경로는 뒤에 정의해야 함

[실습]

경로 파라미터 적용하기

1. routes/user.js - 코드 추가

```
import express from 'express';
const router = express.Router();

// GET /user/
router.get('/', (req, res) => {
  res.send('사용자 목록을 보여줍니다.');
```



```
// GET /user/profile
router.get('/profile', (req, res) => {
  res.send('사용자 프로필 페이지입니다.');
```



```
// POST /user/register
router.post('/register', (req, res) => {
  res.send('사용자 등록이 완료되었습니다.');
```



```
// GET /user/:name → 이름을 경로에서 받기
router.get('/:name', (req, res) => {
  const userName = req.params.name;
  res.send(`${userName}님, 반갑습니다!`);
});
```



```
export default router;
```

2. 동작 확인

요청 주소	응답 내용
/user/nodejs	nodejs님, 반갑습니다!
/user/홍길동	홍길동님, 반갑습니다!

□ 라우터 모듈 추가

- 상품 관련 요청 기능 추가
- 사용자 관련 요청과 상품 관련 요청은 역할이 다르기 때문에 파일로 분리
- 라우트 수가 많아질수록 유지보수가 어려워지기 때문에 기능 단위로 분리된 라우터 모듈을 사용

[실습]

1. 파일 구성

project/

├ routes/

| └ user.js → 사용자 라우터

| └ product.js → 상품 라우터

└ app.js → 라우터 등록 및 서버 실행

2. 라우터 모듈 - routes/product.js

```
import express from 'express';
```

```
const router = express.Router();
```

```
// GET /product → 상품 목록 응답
```

```
router.get('/', (req, res) => {
```

```
  res.send('상품 목록입니다.');
```

```
});
```

```
// GET /product/:id → 특정 상품 상세 정보
```

```
router.get('/:id', (req, res) => {
```

```
  const id = req.params.id;
```

```
  res.send(`상품 상세 페이지 - ${id}번 상품`);
```

```
});
```

```
export default router;
```

3. 라우터 등록 및 실행 - app-route.js

```
import express from 'express';
import userRouter from './routes/user.js';
import productRouter from './routes/product.js';

const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// /user로 시작하는 요청은 userRouter에게 위임
app.use('/user', userRouter);

// /product로 시작하는 요청은 productRouter에게 위임
app.use('/product', productRouter);

app.get('/', (req, res) => {
  res.send('홈 페이지입니다.');
```

```
});

app.listen(3000, () => {
  console.log('http://localhost:3000 서버 실행 중');
```

```
});
```

4. 동작 확인

요청 주소	설명
/product	상품 목록 응답
/product/123	123번 상품 상세 페이지
/product/abc	abc번 상품 상세 페이지 (id로 인식됨)

연습문제

1. 사용자 인사 서버 만들기 (`greet.js` / `app-greet.js`)

- 설명: `/greet/:name` 경로로 요청이 오면 이름으로 인사하고 나이 정보가 함께 오면 나이까지 포함한 문장으로 응답하는 서버를 작성합니다.

- 요구사항

- `greet.js`에서 `express.Router()` 사용
- `app-greet.js`에서 해당 라우터 모듈을 `/greet` 경로에 연결
- `GET /greet/:name` 요청 시
 - `age` 쿼리 값이 있으면 → "안녕하세요, `OOO`님! `N`살이시군요." 응답
 - `age` 쿼리 값이 없으면 → "안녕하세요, `OOO`님!" 응답
- 이름이 없는 경로(`GET /greet`)는 `404`로 처리

- 예상 동작

요청 → `/greet/nodejs`

응답 → `<h2>안녕하세요, nodejs님!</h2>`

요청 → `/greet/nodejs?age=25`

응답 → `<h2>안녕하세요, nodejs님! 25살이시군요.</h2>`

요청 → `/greet`

응답 → `404` (페이지를 찾을 수 없습니다)

- 힌트

- `express.Router()`로 라우터 객체를 만들고 `export default`로 내보냅니다.
- 경로 파라미터는 `router.get('/:name', (req, res) => { ... })` 형태로 정의하고 `req.params.name`으로 값을 꺼냅니다.
- 쿼리스트링은 `req.query.age`로 꺼냅니다.
- `res.send(내용)` : Express에서는 `writeHead+end` 대신 `send()` 하나로 응답을 보낼 수 있습니다.

연습문제

1. 사용자 인사 서버 만들기 (greet.js / app-greet.js)

[greet.js]

// /greet/:name 요청을 처리하는 라우터 모듈

```
import express from 'express';
```

```
const router = express.Router();
```

// 코드 작성 1) GET /:name 라우트 작성하기

// router.get('/:name', (req, res) => { ... })

// 함수 안에서 할 일:

// - req.params.name으로 이름 꺼내기

// - req.query.age로 나이 꺼내기 (age는 안 넘어오면 undefined)

// - age가 있으면: `

##

// - age가 없으면: `

##

// - res.send(내용)으로 응답

```
export default router;
```

[app-greet.js]

// greet.js 라우터를 불러와서 /greet 요청을 처리하는 서버

```
import express from 'express';
```

// 코드 작성 2) greet.js에서 라우터 불러오기

// import 이름 from './파일명.js'

```
const app = express();
```

// 코드 작성 3) '/greet' 경로로 들어오는 요청을 greetRouter에게 위임하기

// app.use('/경로', 라우터)

```
app.listen(3000, () => {
```

```
  console.log('http://localhost:3000 서버 실행 중');
```

```
});
```

연습문제

2. 응답 시간 측정 미들웨어 만들기 (timer.js / app-timer.js)

- 설명: 요청이 들어온 시각을 기록해두었다가 응답이 끝난 시점에 처리 시간(ms)을 콘솔에 출력하는 미들웨어를 작성합니다.

- 요구사항

- timer.js에서 timer(req, res, next) 미들웨어 함수 작성 후 export
 - 미들웨어가 실행되는 시점의 시각을 기록
 - res의 finish 이벤트가 발생하면, 기록해둔 시각과 현재 시각의 차이(ms)를 계산해서 콘솔에 출력
 - next()를 호출해서 다음 미들웨어/라우트로 넘어가기
- app-timer.js에서 app.use(timer)로 모든 요청에 적용

- 예상 동작

(콘솔 출력)

GET / - 3ms

[timer.js]

```
// 코드 작성 1) timer 미들웨어 함수 작성 후 export하기
// export function timer(req, res, next) { ... } (logger 미들웨어 패턴 참고)
// 함수 안에서 할 일:
// - Date.now()로 요청이 들어온 시각을 startTime에 저장
// - res.on('finish', () => { ... }) 등록하기 (2강 app-res-event.js 참고)
// - finish 콜백 안에서: Date.now()로 현재 시각을 구하고 startTime을 빼서 걸린 시간(ms) 계산
// - `${req.method} ${req.url} - ${걸린시간}ms` 형태로 콘솔에 출력
// - next() 호출하기 (다음 미들웨어/라우트로 진행)
```

[app-timer.js]

```
import express from 'express';
// 코드 작성 2) timer.js에서 timer 함수 불러오기
// import { 함수이름 } from './파일명.js' (app-greet.js에서 greetRouter 불러왔던 방식 참고)

const app = express();

// 코드 작성 3) 모든 요청에 timer 미들웨어 적용하기
// app.use(미들웨어함수)

app.get('/', (req, res) => {
  res.send('홈페이지입니다. ');
});

app.listen(3000, () => {
  console.log('http://localhost:3000 서버 실행 중');
});
```

연습문제

3. 상품 라우터 만들기 (product.js / app-product.js)

- 설명: 상품 목록을 카테고리별로 필터링해서 보여주고 특정 상품 하나의 상세 정보도 조회할 수 있는

라우터를 작성합니다.

- 사전 준비 데이터

```
const products = [  
  { id: 1, name: '키보드', category: '전자기기' },  
  { id: 2, name: '마우스', category: '전자기기' },  
  { id: 3, name: '노트', category: '문구' },  
  { id: 4, name: '볼펜', category: '문구' },  
];
```

- 요구사항

- product.js에서 express.Router() 사용
- GET /product 요청 시
 - category 쿼리 값이 없으면 → 전체 상품 이름을 나열해서 응답
 - category 쿼리 값이 있으면 → 해당 카테고리의 상품 이름만 걸러서 응답
- GET /product/:id 요청 시
 - 해당 id를 가진 상품의 이름을 응답
 - 존재하지 않는 id면 "상품을 찾을 수 없습니다" 응답
- app-product.js에서 /product 경로에 라우터 연결

- 예상 동작

요청 → /product

응답 → 키보드, 마우스, 노트, 볼펜

요청 → /product?category=문구

응답 → 노트, 볼펜

요청 → /product/2

응답 → 마우스

요청 → /product/99

응답 → 상품을 찾을 수 없습니다

연습문제

3. 상품 라우터 만들기 (product.js / app-product.js)

```
[product.js]
import express from 'express';
const router = express.Router();

const products = [
  { id: 1, name: '키보드', category: '전자기기' },
  { id: 2, name: '마우스', category: '전자기기' },
  { id: 3, name: '노트', category: '문구' },
  { id: 4, name: '볼펜', category: '문구' },
];

// 코드 작성 1) GET / 라우트 작성하기 (상품 목록, 카테고리 필터링)
// router.get('/', (req, res) => { ... })
// 할 일:
// - req.query.category로 카테고리 값 꺼내기
// - category가 있으면: products.filter(...)로 해당 카테고리만 걸러내기
//   const filtered = category ? products.filter(item => item.category === category) : products;
// - category가 없으면: products 전체 사용
//   걸러진 배열을 .map(item => item.name).join(', ')로 이름만 이어붙여서 응답
//   const names = filtered.map(item => item.name).join(', ');

// 코드 작성 2) GET /:id 라우트 작성하기 (상품 상세)
// router.get('/:id', (req, res) => { ... })
// 주의: 반드시 위의 '/' 라우트보다 아래에 작성해야 합니다
// 할 일:
// - req.params.id를 Number()로 변환
// - products.find(...)로 해당 id를 가진 상품 찾기
// - 찾은 상품이 없으면 (undefined) '상품을 찾을 수 없습니다' 응답
// - 찾았으면 해당 상품의 이름 응답

export default router;
```

```
[app-product.js]
import express from 'express';

// 코드 작성 3) product.js에서 라우터 불러오기

const app = express();

// 코드 작성 4) '/product' 경로로 들어오는 요청을 productRouter에게 위임하기

app.listen(3000, () => {
  console.log('http://localhost:3000 서버 실행 중');
});
```